Lab 10: Testing

Overview

In this lab, we will write unit test cases and design a user acceptance test plan for testing your project. You will work as a group and there needs to be only 1 submission per group. However, if you are absent for this lab without permission, you will receive an irreplaceable "Not attempted" grade for the lab.

To receive credit for this lab, you MUST show your work to the TA during the lab, and push it to the github before the deadline. Please note that submissions will be due on Wednesday at 11:59 p.m. in the following week. ONE SUBMISSION PER GROUP.

Learning Objectives

- ☐ LO1. Distinguish unit, integration, and user acceptance testing.
- ☐ LO2. Write a unit test for an isolated validation function.
- ☐ LO3. Write positive and negative API integration tests using Mocha, Chai, and Chai HTTP.
- ☐ LO4. Verify HTTP responses and database effects using repeatable test data.
- ☐ LO5. Create a UAT plan for at least three project features with measurable acceptance criteria.

Part A

Test Plan

A document describing the scope, approach, resources, and schedule of intended test activities.

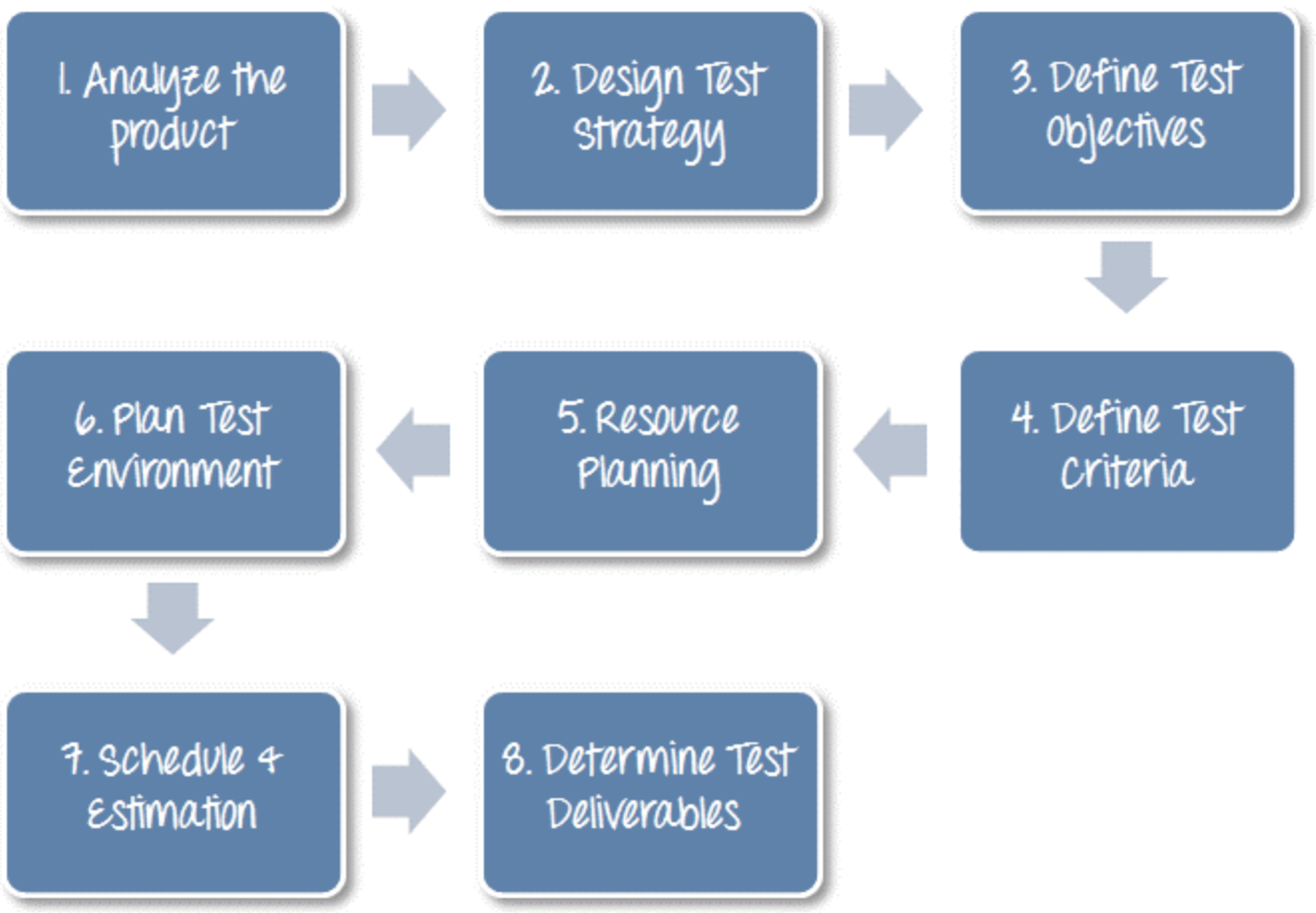
Importance of Test Plan

- To help people outside the test team including developers, business managers, and customers understand the details of testing.
- It guides our thinking. It is like a rule book that we can follow.

- It documents important aspects of testing like test estimation, test scope, and test strategy so it can be reviewed by the management team and re-used for other projects.

Steps to create a Test Plan

The diagram below provides an overview of how to create a Test Plan.



Types of Testing

1. Unit Testing

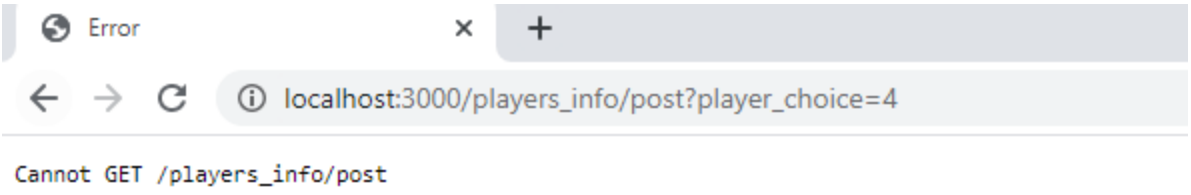
A unit test checks a small piece of code in isolation, such as a function that validates registration input. It should not need an HTTP server or a real database.

2. Integration Testing

Integration testing checks whether components work together. In this lab, HTTP requests pass through Express routes and interact with PostgreSQL. These are **API integration tests**, rather than isolated unit tests.

Type	Example	What is involved?
Unit	Reject an empty password in a validation function	A function and its input
API integration	Register a user and verify the saved record	HTTP, Express, validation, and PostgreSQL
User acceptance	A representative user creates an account and understands the feedback	A user completing a task through the interface

An integration error can occur when one developer uses the form action **'/players_info/post'** while another implements **'/player_info/post'** on the server.



> **TIME TO THINK**
>
> Which route names differ? A function-level unit test may pass even though the form submits to the wrong endpoint. Testing component interactions helps expose this problem.

We will use these tools:

- **Mocha**: Runs tests and reports their results.
- **Chai**: Provides assertions that compare actual behavior with expected behavior.
- **Chai HTTP**: Sends requests to an Express application and supports HTTP assertions.

TODO: Write a unit test and 2 API integration tests

Complete the setup below in your project repository. Preserve your existing application features, dependencies, and development database.

1. Navigate to your project folder

Work in **ProjectSourceCode/**. Paths in the examples are relative to this folder:

- `app.js`: Express configuration, middleware, and routes.
- `index.js`: Starts the application.
- `src/db.js`: Shared database connection.
- `src/validateRegistration.js`: Registration validation function.
- `src/init_data/create.sql`: Your project's database schema.
- `test/`: Automated tests.
- `docker-compose.test.yaml`: Isolated test services.
- `.env.test`: Local test configuration.

Your project may use a different structure. Adapt import paths consistently; do not create duplicate application or database instances.

2. Initialize project and update `package.json`

If `package.json` does not exist, initialize it:

```
npm init -y
```

Keep your existing dependencies and scripts. The following examples use CommonJS (`require` and `module.exports`) with Chai 4 and Chai HTTP 4. Install this explicit combination, rather than installing a newer major version and mixing APIs:

```
npm install --save-dev --save-exact mocha@10.8.2 chai@4.5.0 chai-http@4.4.0
```

The application examples also use Express, `pg-promise`, and `bcryptjs`. Keep your project's installed versions and existing password-hashing library. If any are missing, install them using your course project's dependency versions.

Merge these entries into your existing `scripts` object; this is a fragment, not a replacement for the whole file:

package.json (scripts fragment)

```
{
  "scripts": {
    "start": "node index.js",
    "test": "mocha --timeout 10000 \"test/**/*.spec.js\""
  }
}
```

Keep an existing equivalent `start` script if you already have one. Remove an installation-only `prestart` hook such as `npm install`; install dependencies explicitly during setup. Commit `package.json` and `package-lock.json` so teammates use the same resolved versions. Do not add `--exit` to hide connections that have not been closed.

> **MODULE COMPATIBILITY**
>
> These snippets assume your project uses CommonJS. If it uses `"type": "module"`, use consistent ES module imports/exports with your TA's help; do not change the module type of the entire application just to copy a snippet.

3. Set up docker-compose.test.yaml

Keep your existing development Compose file. Create a **separate** `docker-compose.test.yaml` for tests. For a Docker Compose refresher, see **Lab 1** .

docker-compose.test.yaml

```
services:
  db-test:
    image: postgres:17.6
    env_file: .env.test
    healthcheck:
      test:
        ['CMD-SHELL', 'pg_isready -U "$${POSTGRES_USER}" -d "$${POSTGRES_DB}"']
      interval: 2s
```

```
      timeout: 5s
      retries: 20
      start_period: 10s
    volumes:
      - lab10-test-db:/var/lib/postgresql/data
      - ./src/init_data:/docker-entrypoint-initdb.d:ro
  tests:
    image: node:22.19.0-bookworm
    working_dir: /repository
    env_file: .env.test
    depends_on:
      db-test:
        condition: service_healthy
    volumes:
      - ./:/repository
      - lab10-test-node-modules:/repository/node_modules
    command: ['npm', 'test']
volumes:
  lab10-test-db:
  lab10-test-node-modules:
```

These version tags provide an explicit example baseline. Use the same Node major version across your team; if your project requires another supported version, agree on and document that version with your TA. The PostgreSQL test version should match your development database's major version; if you change it, also use the data-directory mount appropriate to that version.

`depends_on` alone only establishes startup order. The health check makes the test service wait until PostgreSQL can accept connections. No application port or database port needs to be published for these tests.

Use your **actual project schema** in `src/init_data`. If your schema is elsewhere, change the mount path to match. Initialization scripts run when the database volume is first created, not on every test run. Keep development-only seed data and external API calls out of test initialization.

4. Set up .env.test and the database connection

Create this local file:

.env.test

```
NODE_ENV=test
POSTGRES_HOST=db-test
POSTGRES_PORT=5432
POSTGRES_USER=lab10_test
POSTGRES_PASSWORD=local-test-password
POSTGRES_DB=project_test
SESSION_SECRET=local-test-session-secret
```

Use these credentials only for the disposable test services. Add `.env.test` to `.gitignore` and commit a `.env.test.example` with placeholders and setup instructions instead. Keep your existing development `.env`.

Both routes and tests must import the **same database module**. Adapt your existing module to read environment variables rather than hard-coding the development database. The following example uses `pg-promise`:

src/db.js

```
const pgp = require('pg-promise')();

const database = process.env.POSTGRES_DB;
if (
  process.env.NODE_ENV === 'test' &&
  (!database || !database.endsWith('_test'))
) {
  throw new Error('Tests require a database name ending in _test');
}

const db = pgp({
  host: process.env.POSTGRES_HOST,
  port: Number(process.env.POSTGRES_PORT || 5432),
  database,
  user: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD,
});

module.exports = {db, pgp};
```

Update your routes to use this module and preserve your development configuration. If you use `dotenv`, load it without overriding the environment supplied by Compose. Also check for a hard-coded connection URL or a session store that could still point to development resources.

> **⚠ TEST DATA ISOLATION**
>
> Use the isolated test database only. The examples delete specific test-created users; they do not truncate your users table. Never point tests at a shared development or deployed database. The database-name check is an additional safeguard, not a replacement for checking your connection settings.

5. Create and initialize server.spec.js

Create `test/server.spec.js`:

test/server.spec.js

```
const chai = require('chai');
const chaiHttp = require('chai-http');
const app = require('../app');
const {db, pgp} = require('../src/db');
const bcrypt = require('bcryptjs');

chai.use(chaiHttp);
const {expect} = chai;

// Root hook: close the shared database pool after all tests in this process.
// Keep other suites in this file or reuse these imports in separate files.
after(() => {
  pgp.end();
});

describe('Welcome route', () => {
  it('returns the default welcome message', async () => {
    const res = await chai.request(app).get('/welcome');
    expect(res).to.have.status(200);
    expect(res.body).to.deep.equal({
      status: 'success',
      message: 'Welcome!',
    });
  });
```

```
});

// Add your registration tests below.
```

Mocha waits for an `async` test's promise to settle. `await` ensures each assertion runs after its request or database query finishes. A failed assertion throws an error and fails the test. Do not also use a `done` callback in an `async` test.

> **Know before you proceed!**

6. Adding sample route to `app.js`

Move your existing Express setup and routes into `app.js`, preserving their order and behavior. Add this route before any catch-all or error handler:

app.js (route fragment)

```
app.get('/welcome', (req, res) => {
  res.json({status: 'success', message: 'Welcome!'});
});
```

Keep your existing body parsers, view engine, static files, and session middleware. JSON requests need `express.json()`; HTML form requests need `express.urlencoded(...)`.

7. Using module.exports

At the end of `app.js`, export the configured Express application:

app.js (final statement)

```
module.exports = app;
```

Remove `app.listen(...)` from `app.js`. Start the server only in `index.js`:

index.js

```
// Keep any existing environment-loading code before importing app.
const app = require('./app');
const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

You are exporting the **Express application**, which contains its middleware and routes. Chai HTTP can start it on an available port for each request and close that temporary server afterward. Importing `app.js` should not start a separate server on port 3000.

If your application opens other resources, such as a database-backed session store, add the appropriate cleanup hook. Closing an HTTP agent does not close database pools or invalidate sessions stored on the server.

8. Starting Docker Compose

Run these commands from `ProjectSourceCode/`. The explicit project name and file keep test resources separate from your normal development stack.

Install dependencies into the test service's own `node_modules` volume:

```
docker compose -p lab10-tests -f docker-compose.test.yaml run --rm --no-deps
tests npm ci
```

Then run the tests:

```
docker compose -p lab10-tests -f docker-compose.test.yaml up --abort-on-
container-exit --exit-code-from tests
```

Use your existing `docker compose up` workflow separately when you want to run the website. Test execution should finish and return an exit code; it should not launch a development server afterward.

9. Shutting down containers

Stop and remove the test containers while retaining their volumes:

```
docker compose -p lab10-tests -f docker-compose.test.yaml down
```

Use `docker compose down` for normal shutdown of your separate development stack.

10. Check your terminal

The terminal should show the welcome test passing. This screenshot illustrates the style of Mocha output; suite names and totals will differ:

```
> testandrun
> npm-run-all test dev


> test
> mocha

Server is listening on port 3000


  Server!
Database connection successful
    ✓ Returns the default welcome message


  1 passing (40ms)


> dev
> nodemon server.js

[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node server.js`
Server is listening on port 3000
Database connection successful
```

A passing welcome test confirms that the test harness can reach the application. It does not establish that registration or database persistence works.

11. Re-running tests and resetting test data

After editing a test or route, run the test command from Step 8 again. The bind mount makes source edits available without rebuilding or deleting the database. If dependencies change, rerun the `npm ci` command first.

If you intentionally need to reinitialize the test schema, use:

> **RESETTING THE TEST DATABASE**

> The following command deletes **all volumes belonging to the lab10-tests stack**, including test database contents and test dependencies. Use it only to reset disposable test resources. Do not run a volume-deleting command against your development stack.

```
docker compose -p lab10-tests -f docker-compose.test.yaml down --volumes
```

After a reset, rerun both commands in Step 8. For troubleshooting, inspect:

```
docker compose -p lab10-tests -f docker-compose.test.yaml logs db-test tests
```

See the **Docker debugging guide** for more information.

12. Write a unit test and 2 tests for the `/register` API

First, define your registration contract. Record the route, request format, required fields, success response, invalid-input response, and database columns in `test/README.md`. Use the intended behavior of your application to determine the assertions. Do not change behavior simply to make an arbitrary example pass.

The examples below assume:

Item	Example contract
Endpoint	`POST /register`
Request	JSON containing `username` and `password`
Valid username	A nonempty string
Valid password	A string of at least 8 characters
Success	`201` and `{"message":"Registration successful"}`
Missing password	`400` and `{"message":"Invalid input"}`

Item	Example contract
Persistence	One row in `users`; `password` stores a bcrypt hash

Adapt field names, additional required fields, status codes, response assertions, and SQL to your documented contract. If your route expects an HTML form, use `.type('form').send(...)`. If it redirects, disable redirect following and assert its status and `Location` header. The same registration behavior must work through the actual application.

i. Unit test: validate input without HTTP or a database

Extract a small validation function and use it in the registration route before hashing or inserting. Here is an example for the contract above:

src/validateRegistration.js

```
function validateRegistration(input) {
  return Boolean(
    input &&
      typeof input.username === 'string' &&
      input.username.trim().length > 0 &&
      typeof input.password === 'string' &&
      input.password.length >= 8
  );
}

module.exports = validateRegistration;
```

Add at least one unit test. This example checks a missing password:

test/validation.spec.js

```
const {expect} = require('chai');
const validateRegistration = require('../src/validateRegistration');

describe('Registration validation', () => {
  it('rejects a missing password', () => {
    expect(validateRegistration({username: 'student'})).to.equal(false);
  });
```

```
});
```

This is a unit test because it calls one function without HTTP or a database. Testing a validation function your registration route never uses would not protect the real registration workflow.

ii. API integration tests: arrange, act, assert, and clean up

Your project must already have a working registration route and its real schema in `src/init_data/create.sql`. Reuse your existing implementation; do not replace the schema with a simplified example just to pass these tests.

Add the following suite below the welcome suite in `test/server.spec.js`, using the imports already provided. It illustrates repeatable test data and a positive case. Adapt it, then implement the negative case yourself.

test/server.spec.js (registration suite)

```
describe('POST /register', () => {
  const {randomUUID} = require('node:crypto');
  const testUser = {
    username: `lab10_${randomUUID()}`,
    password: 'TestPassword123!',
  };

  async function removeTestUser() {
    await db.none('DELETE FROM users WHERE username = $1', [testUser.username]);
  }

  beforeEach(removeTestUser);
  afterEach(removeTestUser);

  it('creates exactly one user for valid input', async () => {
    // Act: submit the request through the actual registration route.
    const res = await chai.request(app).post('/register').send(testUser);

    // Assert: verify both the response and the saved record.
    expect(res).to.have.status(201);
    expect(res.body.message).to.equal('Registration successful');

    const rows = await db.any(
      'SELECT username, password FROM users WHERE username = $1',
```

```
      [testUser.username]
    );
    expect(rows).to.have.lengthOf(1);
    expect(rows[0].username).to.equal(testUser.username);
    expect(rows[0].password).not.to.equal(testUser.password);
    expect(await bcrypt.compare(testUser.password, rows[0].password)).to.equal(
      true
    );
  });

  // TODO: Add an async test that submits this username without a password.
  // Assert the documented rejection response and zero matching database rows.
});
```

Use your actual hashing library if it is not bcrypt. Salted password hashes differ between runs; verify a password with the library's comparison function rather than comparing two independently generated hashes. Do not send stored password hashes back in API responses.

Positive test case: valid input completes the intended operation.

Negative test case: invalid input is rejected and the operation is not completed. A negative test **passes** when rejection is correct. Use a specific invalid condition, such as a missing password, rather than an unspecified "invalid username and password."

▸ **Choosing the expected HTTP status code**

TO-DO: Complete and verify your tests

1. Test at least one validation rule in isolation and use that function in your route.
2. Write one positive registration integration test. Assert the documented response, exactly one matching row, relevant saved fields, and correct password storage.
3. Write one negative registration integration test. Assert the documented rejection response and that no matching user was inserted.
4. Run the suite twice without manually changing database records. Both runs must pass, including the welcome test.
5. Record setup commands, your route contract, and cleanup behavior in `test/README.md`.

If using test-driven development, define the requirement, write a test that fails because the behavior is missing, implement the behavior, and refactor while keeping the tests passing.

13. Examples for testing Redirect and Render

These are optional patterns to adapt to existing routes, rather than additional required endpoints. The examples use the imports in `server.spec.js`.

i. Testing a redirect:

test/server.spec.js (optional redirect example)

```
describe('Login redirect', () => {
  it('redirects an unauthenticated page request to /login', async () => {
    const res = await chai
      .request(app)
      .get('/dashboard') // Replace with your existing protected page.
      .redirects(0);

    expect(res).to.have.status(302);
    expect(res).to.have.header('location', '/login');
  });
});
```

`.redirects(0)` prevents the client from following the redirect to the final page. Check the original response's status and `Location` header. Adapt the expected location if your contract uses an absolute URL or a return-path query parameter.

ii. Testing a rendered page:

test/server.spec.js (optional render example)

```
describe('Login page', () => {
  it('renders the login form', async () => {
    const res = await chai.request(app).get('/login');
    expect(res).to.have.status(200);
    expect(res).to.be.html;
    // Adapt this marker to your actual login form.
    expect(res.text).to.include('id="login-form"');
  });
```

```
});
```

An HTML response alone could be the wrong page. Check content specific to the expected page as well. This HTTP test does not verify visual layout or browser-side JavaScript.

Part B

3. User Acceptance Testing

User acceptance testing evaluates whether representative users can accomplish the tasks the application is intended to support. It focuses on user goals and observable results. Plan UAT early and execute it when the relevant features are ready; it does not have to be a single final phase.

Below are sample user-facing outcomes:

- A new user can create an account and then sign in.
- A user receives clear feedback when required registration information is missing.
- A user who enters invalid login credentials cannot access a protected feature.

Acceptance Criteria:

Acceptance criteria are observable conditions that must be met for a user story to be accepted. For example:

- Submitting registration without a password displays a clear message identifying the missing field.
- Submitting valid registration information displays confirmation or navigates to the documented next page.
- A newly registered user can sign in using the credentials they just created.

Verifying a specific database table or column is an integration-test concern. UAT checks the behavior the user can observe.

TODO: Create UAT plans for at least 3 features

> **PLAN NOW; RECORD OBSERVATIONS AFTER EXECUTION**

> Submit the plan in this lab. Execute it during the project testing milestone, using the date specified in your project schedule. Actual results and observations are due with the final project report, not this lab. Leave these fields as **Not run** until testing occurs.

Create **one team document at** `milestones/uat-plan.md`, relative to the repository root. Include:

1. **Scope:** At least three project features, linked to user stories or requirements.
2. **Acceptance criteria:** Specific, measurable user-facing outcomes for each feature.
3. **Environment:** Localhost or deployed environment, setup instructions, browser/device, and application version or commit to be tested. Record the actual version when executing.
4. **Testers:** Intended user characteristics, who will recruit or coordinate testers, and planned tester IDs. Choose representative users outside the implementation team when feasible; document limitations if classmates must act as proxies. Use IDs rather than unnecessary personal information.
5. **Test cases:** At least one successful-task case and one invalid-input or alternate-path case for each feature. Include preconditions, exact test data, numbered actions, and observable expected results.
6. **Results fields:** Actual results, status, issues, and follow-up actions. These may remain **Not run** for this lab.

Use this template, with one row per test case:

ıre / ement	Acceptance criterion	Preconditions	Test data	User steps	Expected result	Tester ID	A r
ation int n	Required fields are identified	Registration page is open; user is signed out	Unique username; password left blank	1. Enter username. 2. Leave password blank. 3. Submit.	Registration does not complete; feedback identifies the missing password.	Planned: T1	N ru

The example row illustrates the format; it does not satisfy the requirement for three features. Your expected results should be detailed enough that another tester can determine whether each test passed.

After execution: Record what actually happened, mark each case Pass, Fail, or Blocked, describe issues, and record the outcome of retesting fixes. Include the completed plan and observations in the final project report. Do not invent observations for tests you have not run.

Resources

- **Unit Testing**
- **UAT**
- **Acceptance Criteria**
  - **Mocha**
  - **Chai**

Extra Credit

Write 2 additional API integration tests as a team

Write one positive and one negative integration test for a project feature other than registration. Both must pass and test meaningful behavior beyond the supplied welcome test. Choose scenarios such as authenticated profile access versus unauthenticated access, or valid versus invalid creation of a project-specific record.

If your application uses `express-session`, a Chai HTTP agent can preserve session cookies between login and subsequent requests. This performs a **real login**, not a mocked login.

For example, a profile API may have this contract. Add or adapt it in your existing route structure, after session middleware. Ensure earlier authentication middleware allows this route to return its documented response; otherwise, test the response your middleware actually defines.

app.js (optional profile route)

```
app.get('/profile', (req, res) => {
  if (!req.session.user) {
```

```
    return res.status(401).json({message: 'Not authenticated'});
  }
  return res.status(200).json({username: req.session.user.username});
});
```

This example assumes `/login` accepts JSON and returns `200` with `{"message":"Login successful"}` while setting a session cookie. Adapt the login assertions to your actual contract; if login redirects, disable redirect following and assert the expected redirect.

Add the following to `test/server.spec.js`, reusing its imports and final database-pool cleanup hook:

test/server.spec.js (optional profile suite)

```
describe('GET /profile', () => {
  const {randomUUID} = require('node:crypto');
  const testUser = {
    username: `lab10_profile_${randomUUID()}`,
    password: 'TestPassword123!',
  };
  let agent;

  before(async () => {
    const hashedPassword = await bcrypt.hash(testUser.password, 10);
    await db.none('INSERT INTO users (username, password) VALUES ($1, $2)', [
      testUser.username,
      hashedPassword,
    ]);
  });

  beforeEach(() => {
    agent = chai.request.agent(app);
  });

  afterEach(async () => {
    if (agent) await agent.close();
  });

  after(async () => {
    await db.none('DELETE FROM users WHERE username = $1', [testUser.username]);
  });
```

```
  it('rejects an unauthenticated request', async () => {
    const res = await agent.get('/profile').redirects(0);
    expect(res).to.have.status(401);
    expect(res.body.message).to.equal('Not authenticated');
  });

  it('returns the profile after successful login', async () => {
    const login = await agent.post('/login').send(testUser).redirects(0);
    expect(login).to.have.status(200);
    expect(login.body.message).to.equal('Login successful');
    expect(login).to.have.header('set-cookie');

    const res = await agent.get('/profile').redirects(0);
    expect(res).to.have.status(200);
    expect(res.body).to.deep.equal({username: testUser.username});
  });
});
```

Adapt the fixture insert to include any other required columns. Use a new agent per test so one test cannot inherit another test's login. Agent cleanup closes its temporary HTTP server; clean up any database-backed sessions your project creates separately. If your application uses token-based authentication, test its actual token workflow instead of adding sessions solely for this example.

Submission Guidelines

All work for this lab will be done as a group. Commit the updated application modules, tests, `package.json`, `package-lock.json`, test Compose file, `.env.test.example`, `test/README.md`, and `milestones/uat-plan.md`. Do not commit `.env`, `.env.test`, passwords for real services, or `node_modules`.

Commit and upload your changes

All work for this lab will be done as a group. Commit the updated application modules, tests, `package.json`, `package-lock.json`, test Compose file, `.env.test.example`, `test/README.md`, and `milestones/uat-plan.md`. Do not commit `.env`, `.env.test`, passwords for real services, or `node_modules`.

Submit a link to your GitHub repository to Canvas. Only 1 person per team needs to do this.

Regrade Requests

Please use this **link** to raise a regrade request if you think you didn't receive a correct grade.

Grading Specifications

Criteria	Not Attempted	Below Expectations	Meets Expectations
Overall Attempt	Lab not attempted **OR** student absent without prior approval	—	—
Attendance & TA Review	—	Work not reviewed by TA or individual explanation missing	Attended full lab duration; attendance marked on Canvas; each student explains an assertion; team demonstrates a failing test after an intentional defect and a passing suite after restoration
Repository Submission	—	Required files missing or work not pushed to GitHub or Link not shared on Canvas	Required files committed and pushed; one repository link submitted per team
Unit Test	—	Missing test or test requires HTTP/database	At least one passing test of an isolated validation function used by the registration route
API Integration	—	Positive/negative cases missing, failing,	Both cases pass; success case verifies response, exactly one

Criteria	Not Attempted	Below Expectations	Meets Expectations
Tests (/register)		or missing required assertions	matching row, relevant fields, and password storage; invalid-input case verifies rejection and no inserted user
Repeatable Test Setup	—	Uses development data, lacks cleanup, or requires manual repair between runs	Isolated test database; targeted fixture cleanup; suite passes twice; commands and route contract documented in `test/README.md`
UAT Plan (Presence)	—	Plan missing or stored in the wrong location	One team plan committed at `milestones/uat-plan.md`
UAT Plan (Content Quality)	—	Fewer than three features or required fields/scenarios missing	At least three features, each with acceptance criteria, successful and invalid/alternate cases, preconditions, data, steps, expected results; environment and representative tester plan documented; actual results may be Not run
Extra Credit – Automated Tests	—	—	Meets all core expectations and adds two passing integration tests, one positive and one negative, for a feature other than registration